ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR
Membre de HONORIS UNITED UNIVERSITIES

CAHIER DES CHARGES

SMART UNIVERSITY ASSISTANT

Conception et développement d'un assistant universitaire intelligent basé sur l'intelligence artificielle et la génération augmentée par récupération (RAG)

Module : Développement d'applications intelligentes
Réalisé par : Ettayeb Benchekroune – Ilyass Sekouni – Taha El bal
Encadré par : Inas Hasnaoui
Année universitaire : 2026/2027

1. Présentation générale du projet

1.1. Contexte

Les étudiants rencontrent régulièrement des difficultés pour retrouver rapidement les informations nécessaires à leur parcours universitaire. Ces informations peuvent être dispersées dans plusieurs documents, notamment les règlements intérieurs, les guides pédagogiques, les calendriers universitaires, les documents administratifs et les supports de cours.

La consultation manuelle de ces ressources peut être longue et ne garantit pas toujours que l'information trouvée soit pertinente ou à jour.

L'intelligence artificielle générative, combinée aux techniques de recherche sémantique et de génération augmentée par récupération (Retrieval-Augmented Generation ou RAG), offre la possibilité de développer un assistant capable de comprendre des questions formulées en langage naturel et de produire des réponses fondées sur des documents de référence.

1.2. Présentation de la solution

Smart University Assistant est une application web intelligente destinée à faciliter l'accès aux informations universitaires.

Elle permet aux utilisateurs autorisés de consulter les ressources documentaires et de poser des questions en langage naturel. Le système recherche les passages pertinents dans une base documentaire, fournit le contexte récupéré à un modèle de langage et génère une réponse accompagnée de références permettant de vérifier les informations présentées.

La solution comprend également un espace d'administration, un système d'authentification et de gestion des rôles, une gestion documentaire, un historique des conversations et des mécanismes d'automatisation des traitements.

1.3. Problématique

Comment concevoir une application intelligente capable de fournir aux étudiants des réponses rapides, pertinentes et vérifiables à partir de documents universitaires officiels, tout en assurant la sécurité des données, la traçabilité des réponses et la facilité d'administration de la plateforme ?

1.4. Objectifs

Les objectifs principaux sont les suivants :

- Centraliser les documents universitaires dans une base documentaire.
- Permettre une recherche documentaire sémantique.

- Fournir une interface conversationnelle accessible et simple d'utilisation.
- Générer des réponses à partir des documents récupérés grâce à une architecture RAG.
- Afficher les sources utilisées pour justifier les réponses.
- Limiter les réponses non fondées lorsque les documents disponibles ne contiennent pas l'information recherchée.
- Gérer les comptes utilisateurs et les droits d'accès.
- Automatiser certains traitements documentaires à l'aide de n8n.
- Mesurer les performances du système à travers des tests et des indicateurs.
- Produire une solution documentée, reproductible et présentable.

2. Périmètre fonctionnel

2.1. Fonctionnalités incluses

A. Authentification et gestion des utilisateurs

Le système doit permettre :

- La création d'un compte utilisateur.
- La connexion et la déconnexion.
- La gestion sécurisée des mots de passe.
- La génération et la validation de jetons d'accès JWT.
- La consultation du profil de l'utilisateur connecté.
- La gestion des rôles et des permissions.
- L'activation et la désactivation des comptes.

Trois rôles sont prévus :

Administrateur :

- Gérer les utilisateurs et leurs rôles.
- Ajouter, consulter, modifier et supprimer les documents.
- Consulter les statistiques d'utilisation.
- Superviser les traitements documentaires et les automatisations.
- Consulter les journaux techniques nécessaires à l'administration.

Étudiant :

- Consulter les ressources accessibles.
- Poser des questions à l'assistant intelligent.
- Consulter les réponses et leurs sources.
- Créer et consulter ses conversations.
- Consulter son historique personnel.

Enseignant :

- Consulter les ressources pédagogiques autorisées.
- Interroger l'assistant.
- Consulter les sources des réponses.
- Accéder aux fonctionnalités pédagogiques prévues par les permissions qui lui sont attribuées.

B. Gestion documentaire

L'administrateur doit pouvoir :

- Importer des documents aux formats PDF et TXT.
- Consulter la liste des documents.
- Visualiser les métadonnées des documents.
- Modifier les informations descriptives.
- Supprimer un document et ses données d'indexation associées.
- Consulter le statut du traitement documentaire.
- Identifier les documents en attente, en cours de traitement, traités ou en erreur.

Les métadonnées minimales comprennent le titre, le nom du fichier, le type MIME, la taille, la date d'importation, l'utilisateur ayant effectué l'importation et le statut du traitement.

Les documents doivent provenir de sources universitaires autorisées. Le système doit conserver une référence permettant d'identifier chaque document d'origine.

C. Traitement et indexation des documents

Après l'importation d'un document, le système doit exécuter un pipeline comprenant :

1. Vérification du format, de la taille et de la validité du fichier.

2. Extraction du texte.
3. Nettoyage et normalisation du texte.
4. Découpage du texte en segments, appelés chunks.
5. Génération des embeddings de chaque segment.
6. Enregistrement des segments et de leurs métadonnées.
7. Indexation des embeddings pour permettre la recherche sémantique.
8. Mise à jour du statut du document.

Les fichiers illisibles ou les documents qui ne peuvent pas être traités doivent être signalés avec un message d'erreur explicite.

D. Recherche documentaire sémantique

Le système doit interpréter la signification d'une question et rechercher les segments documentaires les plus pertinents.

Les résultats doivent pouvoir être filtrés selon les documents accessibles à l'utilisateur. La recherche doit retourner les passages sélectionnés, leur pertinence et leurs références documentaires.

E. Assistant conversationnel intelligent

L'assistant doit permettre :

- La saisie de questions en langage naturel.
- La création et la consultation de conversations.
- La génération de réponses à partir du contexte documentaire récupéré.
- L'affichage des sources utilisées.
- La conservation de l'historique des échanges.
- La gestion des erreurs de génération et de recherche.
- Le refus de produire une réponse factuelle non étayée lorsque les sources disponibles sont insuffisantes.

Chaque réponse doit être associée aux sources réellement utilisées pour sa génération. Une source doit permettre de retrouver le document d'origine et, si possible, le passage ou la page concernée.

Le système ne doit pas présenter une information générée par le modèle comme une information officiellement confirmée lorsqu'elle n'est pas soutenue par les documents disponibles.

F. Historique et gestion des conversations

L'utilisateur doit pouvoir :

- Créer plusieurs conversations.
- Consulter ses conversations précédentes.
- Retrouver les messages d'une conversation.
- Consulter les sources associées aux réponses.
- Supprimer ses propres conversations lorsque cette fonctionnalité est autorisée.

Un étudiant ne doit pas pouvoir consulter les conversations privées d'un autre utilisateur.

G. Tableau de bord d'administration

Le tableau de bord doit afficher des indicateurs tels que :

- Nombre total d'utilisateurs.
- Répartition des utilisateurs par rôle.
- Nombre de documents enregistrés.
- Nombre de documents traités, en attente ou en erreur.
- Nombre de conversations et de questions.
- Activité récente.
- État des principaux traitements automatisés.

Ces indicateurs doivent être calculés à partir des données réellement enregistrées.

H. Automatisation avec n8n

n8n doit être utilisé pour orchestrer certains processus automatisés, notamment :

- Le déclenchement ou la coordination d'un traitement documentaire.
- La surveillance du résultat d'un traitement.
- La notification d'une réussite ou d'un échec.
- L'enregistrement d'informations de suivi.
- La gestion de nouvelles tentatives lorsque cela est pertinent.

Les workflows doivent être documentés et exportables. Les secrets et les jetons d'accès ne doivent pas être exposés dans les journaux ou les fichiers exportés.

L'automatisation ne doit pas remplacer les contrôles de sécurité effectués par le backend.

2.2. Fonctionnalités exclues du périmètre initial

Afin de respecter le délai de 30 jours, les éléments suivants ne constituent pas des exigences obligatoires de la première version :

- Intégration à un système d'information universitaire réel.
- Synchronisation automatique avec les emplois du temps institutionnels.
- Application mobile native.
- Paiement en ligne.
- Reconnaissance vocale avancée.
- Entraînement d'un modèle de langage depuis zéro.
- Gestion complète des notes et des inscriptions administratives.
- Déploiement industriel à grande échelle.

Ces fonctionnalités pourront être envisagées comme des évolutions futures.

3. Architecture technique

3.1. Architecture générale

La solution repose sur une architecture modulaire organisée autour des composants suivants :

Interface utilisateur — Streamlit

Elle fournit les interfaces de connexion, de consultation documentaire, de conversation et d'administration.

Backend — FastAPI

Il expose les API REST et centralise la logique métier, les règles d'accès, l'authentification, la gestion des utilisateurs, les documents, les conversations et l'orchestration du pipeline RAG.

Base de données — PostgreSQL

Elle conserve les comptes, les rôles, les métadonnées documentaires, les segments de texte, les conversations, les messages, les références des sources et les données nécessaires aux statistiques.

Composant de recherche sémantique

Il permet de rechercher les segments documentaires les plus pertinents à partir des embeddings. Une extension vectorielle de PostgreSQL ou une solution vectorielle compatible pourra être utilisée selon les ressources disponibles.

Modèle de langage — LLM

Il génère des réponses en utilisant les passages récupérés. Le modèle peut être exécuté localement ou être accessible par une API, selon les ressources matérielles et les contraintes du projet.

Automatisation — n8n

Il orchestre les workflows documentaires, les notifications et le suivi des traitements.

Gestion du code — Git et GitHub

Ils assurent le suivi des modifications, la gestion des versions et la sauvegarde du code source.

3.2. Organisation logique du backend

Le backend doit être structuré en modules distincts :

- api/ : routes et points d'entrée HTTP.
- core/ : configuration, base de données, sécurité et permissions.
- models/ : modèles de données SQLAlchemy.
- schemas/ : schémas de validation Pydantic.
- services/ : logique métier, traitement documentaire et RAG.
- repositories/ : opérations d'accès aux données.

Cette séparation facilite les tests, la maintenance et les évolutions.

3.3. Technologies retenues

- Langage principal : Python.
- Backend : FastAPI.
- Interface web : Streamlit.
- Base de données relationnelle : PostgreSQL.
- ORM : SQLAlchemy.
- Validation des données : Pydantic.
- Authentification : JWT et hachage sécurisé des mots de passe.
- IA générative : modèle LLM local ou accessible par API.

- Embeddings : modèle d'embeddings adapté à la recherche sémantique.
- Recherche vectorielle : solution compatible avec les ressources disponibles.
- Automatisation : n8n.
- Documentation des API : Swagger/OpenAPI.
- Gestion des versions : Git et GitHub.
- Tests : pytest et outils de test des API.

Les versions exactes des dépendances devront être consignées dans le fichier de configuration du projet.

4. Modèle de données prévisionnel

Les principales entités sont les suivantes.

4.1. User

Représente un utilisateur de la plateforme.

Attributs principaux :

- id
- full_name
- email
- hashed_password
- role
- is_active
- created_at

Chaque utilisateur possède un rôle parmi les rôles autorisés.

4.2. Document

Représente un document universitaire importé.

Attributs principaux :

- id
- title
- filename
- file_type

- file_size
- storage_path
- status
- uploaded_by
- created_at
- updated_at

Un document est importé par un utilisateur et peut être découpé en plusieurs segments.

4.3. DocumentChunk

Représente un segment de texte extrait d'un document.

Attributs principaux :

- id
- document_id
- chunk_index
- content
- page_number
- metadata
- created_at

Un segment appartient à un seul document. Un document peut contenir plusieurs segments.

4.4. Conversation

Représente une session de discussion.

Attributs principaux :

- id
- user_id
- title
- created_at
- updated_at

Une conversation appartient à un seul utilisateur et contient plusieurs messages.

4.5. Message

Représente une question ou une réponse.

Attributs principaux :

- id
- conversation_id
- role
- content
- created_at

Le rôle du message indique s'il s'agit d'un message utilisateur, assistant ou, si nécessaire, système.

4.6. MessageSource

Représente le lien entre une réponse et une source documentaire utilisée.

Attributs principaux :

- id
- message_id
- chunk_id
- relevance_score
- citation_text

Une réponse peut être liée à plusieurs sources. Une source documentaire peut être citée dans plusieurs réponses.

4.7. WorkflowExecution

Représente une exécution d'un workflow automatisé.

Attributs principaux :

- id
- workflow_name
- status
- started_at
- finished_at
- error_message

Cette entité facilite le suivi des traitements et le diagnostic des erreurs.

Les tables définitives devront être définies à partir du modèle UML et de l'implémentation réelle. Les embeddings peuvent être conservés dans une colonne vectorielle ou dans un système spécialisé, selon l'architecture choisie.

5. Exigences non fonctionnelles

5.1. Sécurité

- Les mots de passe doivent être hachés avec un algorithme adapté.
- Les clés secrètes doivent être stockées dans des variables d'environnement.
- Les routes protégées doivent vérifier l'identité et les permissions de l'utilisateur.
- Les opérations d'administration doivent être réservées aux administrateurs.
- Les données privées ne doivent pas être accessibles à des utilisateurs non autorisés.
- Les entrées utilisateur et les fichiers importés doivent être validés.
- Les erreurs retournées au client ne doivent pas divulguer d'informations sensibles.

5.2. Fiabilité

- Les erreurs de traitement doivent être détectées et enregistrées.
- Les opérations sur la base de données doivent utiliser des transactions lorsque cela est nécessaire.
- Un document en erreur ne doit pas être considéré comme correctement indexé.
- Les réponses du chatbot doivent être liées aux sources réellement récupérées.
- Les opérations critiques doivent être testées.

5.3. Performance

Le système doit rester utilisable sur la machine de développement disponible. Les temps de traitement documentaire, de recherche et de génération doivent être mesurés séparément.

Les objectifs chiffrés de performance seront définis après les premiers essais, puis comparés aux résultats mesurés.

5.4. Maintenabilité

- Le code doit être organisé en modules.

- Les noms des fonctions, classes et variables doivent être explicites.
- Les dépendances doivent être documentées.
- Les erreurs importantes doivent être journalisées.
- Le projet doit contenir un README décrivant l'installation et le lancement.
- Les principales fonctionnalités doivent être accompagnées de tests.

5.5. Ergonomie

L'interface doit être claire, cohérente et facile à utiliser. Les erreurs de connexion, les états de traitement des documents et les sources des réponses doivent être compréhensibles pour les utilisateurs.

6. Exigences relatives au RAG et à la qualité des réponses

6.1. Pipeline RAG

Le traitement d'une question doit suivre les étapes suivantes :

1. Réception de la question.
2. Vérification des droits d'accès de l'utilisateur.
3. Transformation de la question en représentation vectorielle.
4. Recherche des segments documentaires pertinents.
5. Filtrage et sélection du contexte.
6. Construction du prompt du modèle de langage.
7. Génération de la réponse à partir du contexte disponible.
8. Association de la réponse aux sources utilisées.
9. Enregistrement de la question, de la réponse et des références.
10. Affichage de la réponse dans l'interface.

6.2. Gestion des informations absentes

Si les documents récupérés ne permettent pas de répondre suffisamment à une question, le système doit l'indiquer explicitement et inviter l'utilisateur à consulter le service compétent ou à fournir un document pertinent.

Le modèle ne doit pas inventer une référence documentaire, une page ou une règle universitaire.

6.3. Évaluation expérimentale

Les essais doivent utiliser un ensemble de questions de référence, accompagnées des réponses attendues et des documents pertinents.

Les indicateurs envisagés sont :

- Taux de réponses correctement étayées par les sources.
- Pertinence des passages récupérés.
- Exactitude des réponses par rapport aux références.
- Taux de réponses sans source suffisante correctement refusées.
- Temps moyen de recherche.
- Temps moyen de génération.
- Taux d'erreurs techniques.
- Résultats de tests de sécurité et d'autorisation.

Les valeurs seront mesurées pendant l'expérimentation. Aucune performance ne sera déclarée comme acquise avant d'avoir été vérifiée.

7. Contraintes et hypothèses

- Le projet est réalisé individuellement.
- La durée cible est de 30 jours.
- Le développement est réalisé sur un environnement Windows.
- Les ressources matérielles disponibles peuvent limiter la taille du modèle de langage.
- La qualité des réponses dépend de la qualité, de la pertinence et de l'actualité des documents.
- L'accès aux documents universitaires doit être autorisé.
- Les services externes éventuels doivent être utilisés conformément à leurs conditions d'utilisation.
- La disponibilité d'un modèle local ou d'une API dépendra des ressources et de la configuration retenues.

8. Planification prévisionnelle sur 30 jours

Semaine 1 — Cadrage et fondations techniques

- Jours 1 à 2 : définition des besoins et du périmètre.

- Jours 3 à 4 : diagrammes UML et conception de la base de données.
- Jours 5 à 7 : configuration du projet, PostgreSQL, modèles et authentification.

Livrable : architecture initiale, diagrammes UML et authentification opérationnelle.

Semaine 2 — Gestion des données et des documents

- Jours 8 à 9 : gestion des utilisateurs et des permissions.
- Jours 10 à 11 : importation et gestion des documents.
- Jours 12 à 13 : extraction du texte et découpage en segments.
- Jour 14 : tests du pipeline documentaire.

Livrable : API de gestion documentaire et pipeline d'extraction fonctionnel.

Semaine 3 — Recherche sémantique et assistant IA

- Jours 15 à 17 : embeddings et indexation vectorielle.
- Jours 18 à 19 : recherche sémantique.
- Jours 20 à 21 : intégration du LLM et génération augmentée par récupération.

Livrable : assistant capable de répondre à des questions à partir de documents de référence.

Semaine 4 — Interface, automatisation et validation

- Jours 22 à 24 : interface Streamlit et tableau de bord.
- Jours 25 à 26 : workflows n8n.
- Jours 27 à 28 : tests fonctionnels, sécurité et mesures de performance.
- Jour 29 : documentation, correction des anomalies et préparation du rapport.
- Jour 30 : démonstration finale et bilan expérimental.

Livrable : version intégrée, tests, workflows, documentation et démonstration.

Cette planification est indicative. Les priorités pourront être ajustées selon les difficultés techniques rencontrées.

9. Tests et critères d'acceptation

Le projet sera considéré comme fonctionnel lorsque les critères suivants auront été vérifiés :

1. Un utilisateur peut créer un compte et se connecter.
2. Les mots de passe ne sont pas conservés en clair.

3. Les permissions distinguent correctement les rôles étudiant, enseignant et administrateur.
4. Un utilisateur non autorisé ne peut pas accéder aux fonctions administratives.
5. Un administrateur peut importer et gérer les documents.
6. Les documents valides peuvent être extraits et indexés.
7. Une question peut déclencher une recherche sémantique.
8. L'assistant peut produire une réponse accompagnée de sources vérifiables.
9. Une question non couverte par les documents conduit à une réponse prudente plutôt qu'à une affirmation inventée.
10. Les conversations et les messages sont correctement enregistrés.
11. Les workflows n8n peuvent être exécutés et leurs résultats suivis.
12. Les principales fonctionnalités sont couvertes par des tests.
13. Le projet peut être installé et lancé à partir des instructions du README.

10. Livrables attendus

- Cahier des charges.
- Diagramme de cas d'utilisation.
- Diagramme de classes.
- Diagrammes complémentaires si nécessaires.
- Code source du backend et de l'interface.
- Scripts ou migrations de base de données.
- Pipeline documentaire et composants RAG.
- Workflows n8n exportés.
- Fichier des dépendances.
- Tests automatisés et résultats expérimentaux.
- Documentation d'installation et d'utilisation.
- Rapport présentant l'architecture, les choix techniques, les résultats, les limites et les perspectives.
- Démonstration fonctionnelle du système.

11. Perspectives d'évolution

Les évolutions possibles comprennent :

- Intégration à un portail universitaire officiel.
- Ajout d'autres formats documentaires.
- Prise en charge avancée du français, de l'arabe et de l'anglais.
- Amélioration de la recherche hybride combinant recherche lexicale et recherche vectorielle.
- Gestion des versions et de la validité temporelle des documents.
- Ajout d'analyses statistiques plus avancées.
- Déploiement sur un serveur accessible aux utilisateurs.
- Mise en place d'une stratégie de sauvegarde et de supervision.

12. Conclusion

Smart University Assistant vise à simplifier l'accès à l'information universitaire grâce à une architecture combinant une API modulaire, une base de données relationnelle, une recherche sémantique et un assistant conversationnel basé sur le RAG.

La priorité du projet est de produire une solution fonctionnelle, sécurisée, testable et documentée, capable de répondre à des questions en s'appuyant sur des sources universitaires identifiables. L'expérimentation permettra d'évaluer les performances réelles du système, d'identifier ses limites et de proposer des améliorations pertinentes.